

Industrial Internship Report on Inventory Management System

Prepared by
Sandhya Sapkota

Executive Summary

This report presents Stockroom, a web-based Inventory Management System developed during a Python development internship, following the format provided by upskill Campus and The IoT Academy in association with UCT.

Built using Python, Flask, SQLite, Jinja2, HTML, CSS, and JavaScript, Stockroom replaces manual records with a central product catalog, stock tracking, low-stock indicators, role-based access, and movement history. Administrators manage products and accounts, while staff record stock movements and export reports.

Security measures include password hashing and CSRF protection. Stock updates and movement records are saved in a single database transaction, with validation preventing invalid quantities and negative balances. Only zero-stock products can be archived, preserving their history.

Six automated test methods passed, covering key functionality and data consistency. The application remains a local small-team prototype; production capacity has not been measured. Remaining submission tasks include personal details, implementation review, screenshots, and a demonstration. Future enhancements include public deployment, email alerts, barcode scanning, and supplier workflows.

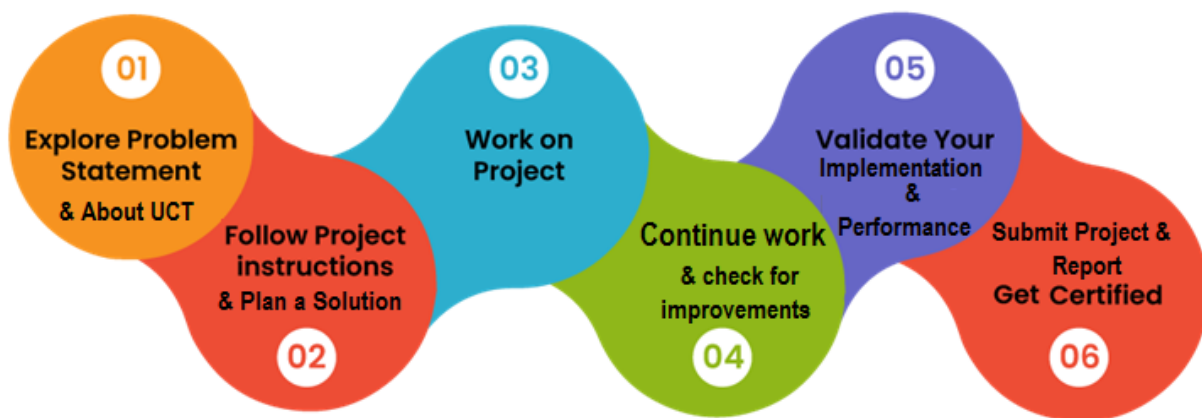
TABLE OF CONTENTS

1	Preface	3
2	Introduction	4
2.1	About UniConverge Technologies Pvt Ltd	4
i.	UCT IoT Platform	4
2.2	About upskill Campus (USC)	8
2.3	The IoT Academy	10
2.3	Objective	10
2.4	Reference.....	10
2.5	Glossary	11
3	Problem Statement.....	12
4	Existing and Proposed solution.....	13
4.1	Existing Approaches.....	13
4.2	Proposed Solution and Value Addition.....	13
4.3	Code submission (Github link).....	13
4.4	Report submission (Github link) :	14
5	Proposed Design/ Model	14
5.1	High-Level Diagram.....	14
5.2	Low Level Diagram.....	15
	Stock Movement Algorithm	15
5.3	Interfaces (if applicable)	16
5.4	Application Screenshots	17
6	Performance Test.....	20
6.1	Test Plan/ Test Cases	20
6.2	Test Procedure.....	21
6.3	Performance Outcome	21
7	My learnings.....	23
8	Future work scope	24

1 Preface

An internship project connects programming concepts with a practical software workflow. Inventory management is a suitable problem because it requires persistent records, reliable validation, controlled access, and traceable changes rather than only a visual interface.

Stockroom was selected as the final project for the Python development domain. Its scope covers product records, stock movements, low-stock alerts, administrator and staff roles, and reporting. The implementation is designed to be understandable and practical for a one-week final-project preparation window.



Acknowledgement: I thank upskill and the internship team for their guidance and support.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



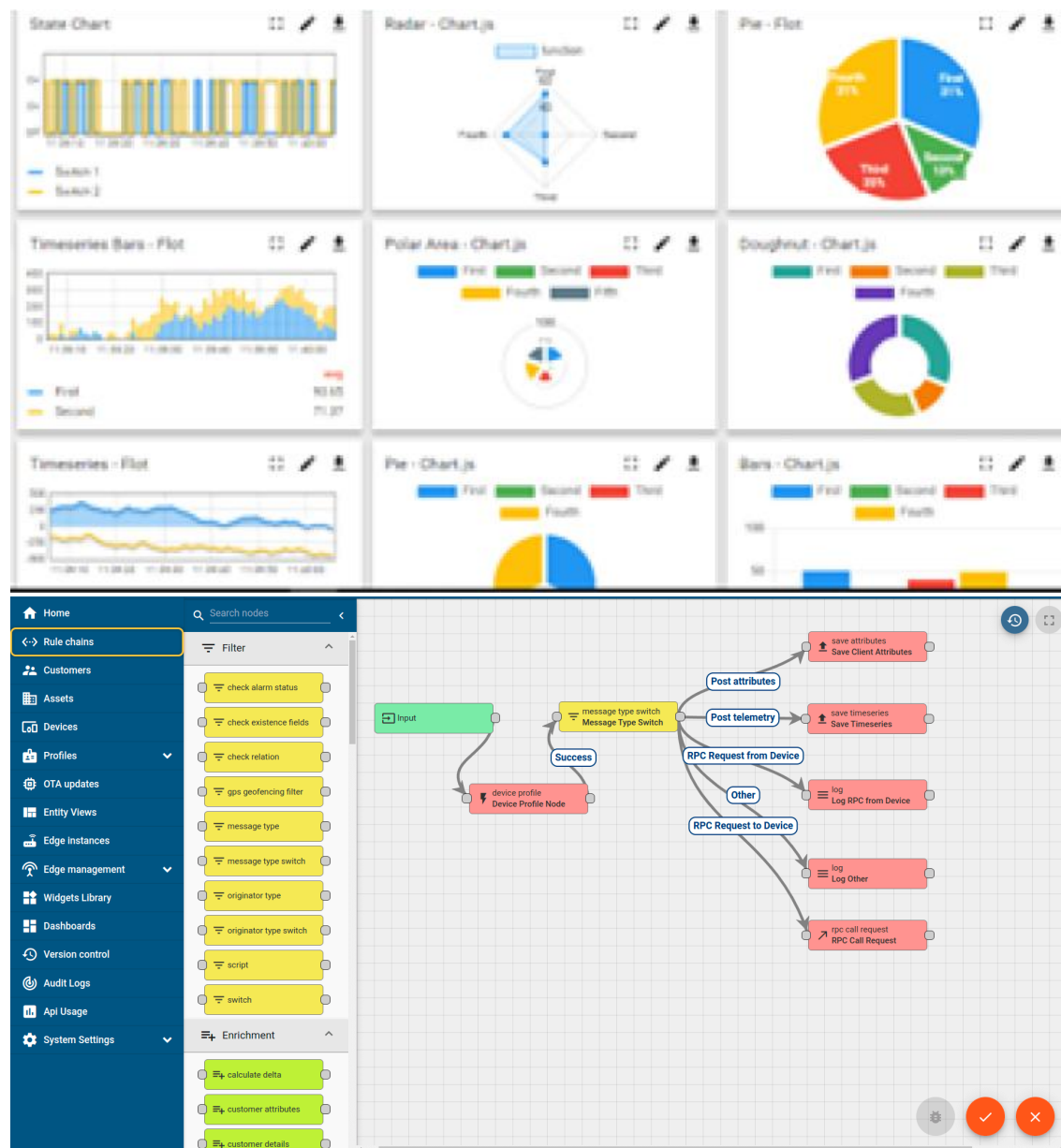
i. UCT IoT Platform ()

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



FACTORY WATCH

ii. Smart Factory Platform ()

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i



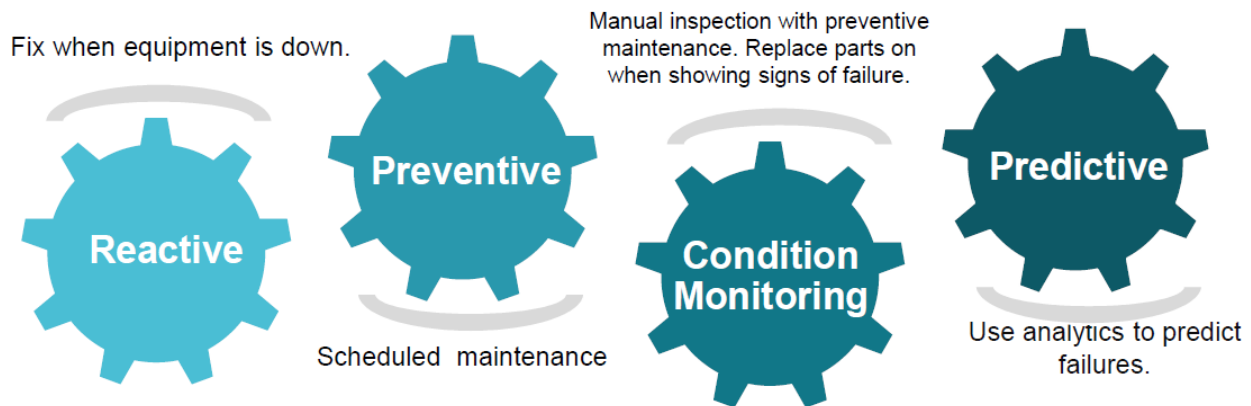


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

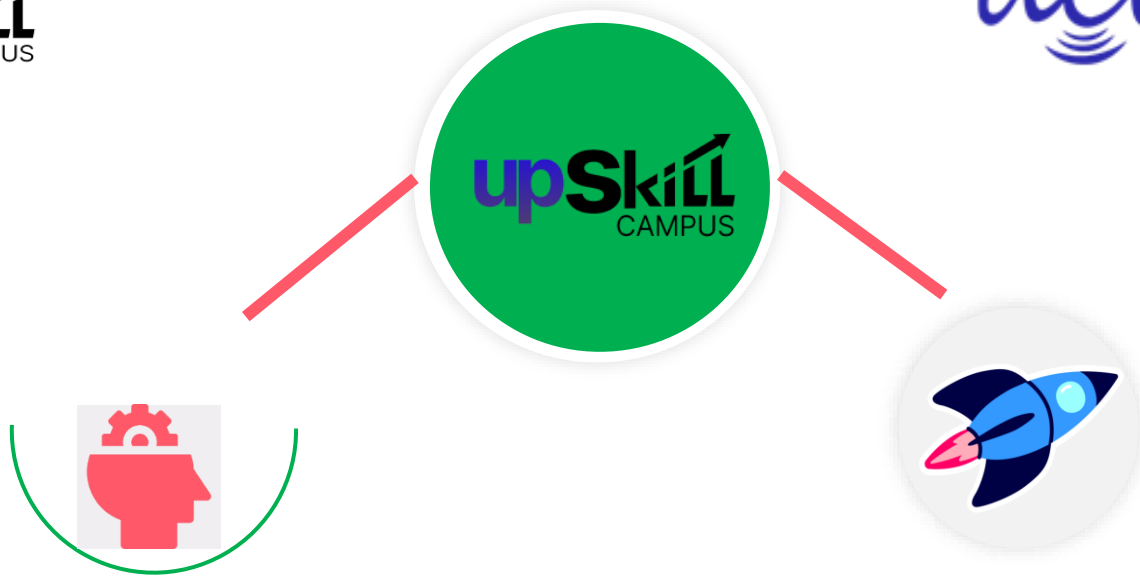
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

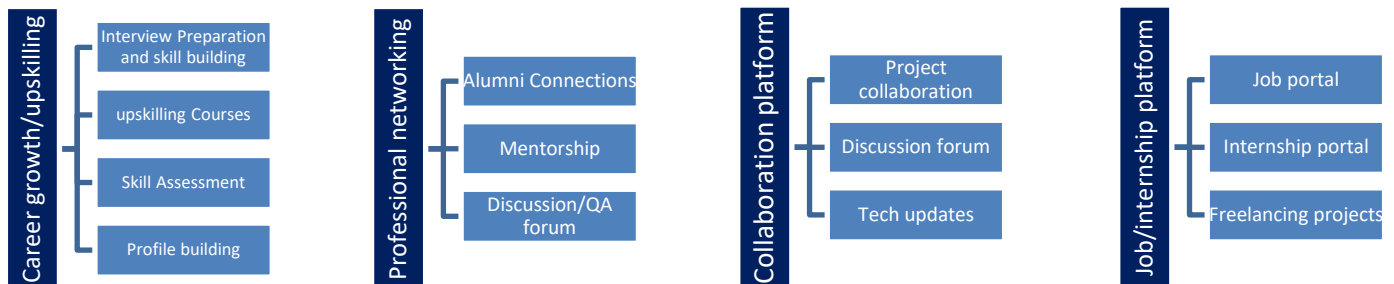
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

<https://www.upskillcampus.com/>



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.3 Objective

The objective is to build and explain a functioning Python web application for managing inventory in a small organization.

- Maintain uniquely identified products with names, categories, unit prices, and reorder levels.
- Record stock receipts and issues without allowing negative inventory.
- Highlight products whose stock is at or below the reorder threshold.
- Enforce administrator and staff access rules on the server.
- Preserve the actor, reason, timestamp, and resulting balance for each movement.
- Provide searchable inventory and a downloadable CSV report.
- Validate important workflows through automated tests and a repeatable demonstration.

2.4 Reference

[1] Company-supplied sample: Sample_InternshipReport_USC_UCT (1).docx.

[2] Flask documentation: <https://flask.palletsprojects.com/en/stable/>

[3] Python sqlite3 documentation: <https://docs.python.org/3/library/sqlite3.html>

[4] Werkzeug security utilities: <https://werkzeug.palletsprojects.com/en/stable/utils/#module-werkzeug.security>

[5] Project source: app.py, templates/, static/, tests/test_app.py, and README.md in the submitted repository.

2.5 Glossary

Term	Meaning in this project
SKU	Stock Keeping Unit; a unique product identifier.
CRUD	Create, read, update, and delete; products use archival instead of destructive deletion.
CSRF	Cross-Site Request Forgery; form tokens help reject unauthorized form submissions.
RBAC	Role-based access control; admin and staff have different permissions.
Transaction	A unit of database work committed or rolled back together.
UTC	Coordinated Universal Time; used for movement timestamps.
CSV	Comma-separated values; the downloadable inventory report format.
Reorder level	Quantity threshold at or below which an alert appears.

3 Problem Statement

A small business needs a reliable way to identify products, see current stock, record incoming and outgoing items, and know when replenishment is required. When changes are tracked manually, users can overwrite records, miss low-stock conditions, or lose the reason for a quantity change.

The proposed system must provide a central source of inventory data and reject changes that violate business rules. Every accepted movement must identify the product, user, movement direction, quantity, resulting balance, optional reason, and time.

Requirement	Acceptance criterion
Product identification	SKU is unique; product name and SKU are mandatory.
Stock accuracy	Only positive whole movement amounts are accepted; stock remains nonnegative.
Traceability	Each accepted movement creates a permanent history record.
Access control	Staff cannot access product administration or user management routes.
Low-stock visibility	Products with quantity $\leq$ reorder level appear in the attention list.
Safe retirement	Only zero-stock products may be archived; previous movements remain visible.
Usable reporting	Users can search active products and export their catalog as CSV.

Scope boundaries: one inventory workspace, whole-unit quantities, a single price convention, and in-app alerts. The app does not process payments, calculate accounting cost layers, send email, or manage multiple warehouse locations.

4 Existing and Proposed solution

4.1 Existing Approaches

Approach	Advantages	Limitations for this project
Paper register	Low setup cost and familiar workflow.	Difficult searching, calculations, role enforcement, and timely alerts.
Spreadsheet	Flexible tables, formulas, and exports.	Manual edits can change quantities without consistent movement history.
Full ERP / inventory platform	Broad purchasing, warehousing, and reporting workflows.	May introduce setup and training beyond a small internship prototype.

These comparisons describe general workflow trade-offs, not a benchmark of any specific commercial product.

4.2 Proposed Solution and Value Addition

Stockroom combines a simple web interface with server-side validation and a transactional SQLite database. Users authenticate before accessing inventory. The dashboard summarizes active products, units, current-price inventory value, and low-stock items.

Products are created with zero quantity. Opening stock is received through the same movement workflow used for later deliveries. This prevents an unexplained starting balance. A permanent movement log and zero-stock-only archival retain an auditable stock history.

Administrators manage catalog records and create accounts. Staff can record movements, view information, and export inventory without changing catalog definitions or creating additional users.

4.3 Code submission (GitHub link)

(GitHub link): [sandhyasapkota/upskillcampus](https://github.com/sandhyasapkota/upskillcampus)

4.4 Report submission (GitHub link):

(GitHub link):

[upskillcampus/docs/InventoryManagementSystem_SandhyaSapkota_USC_UCT.pdf](https://github.com/sandhyasapkota/InventoryManagementSystem) at main · sandhyasapkota/upskillcampus

5 Proposed Design/ Model

5.1 High-Level Diagram

Layer	Components	Responsibility
Presentation	Browser → HTML / CSS / JavaScript	Forms, navigation, dashboard, tables, and confirmation prompts.
Application	Flask routes + Jinja templates	Authentication, permissions, validation, business rules, and rendered responses.
Persistence	SQLite: users, products, movements	Store accounts, balances, catalog attributes, and stock history.

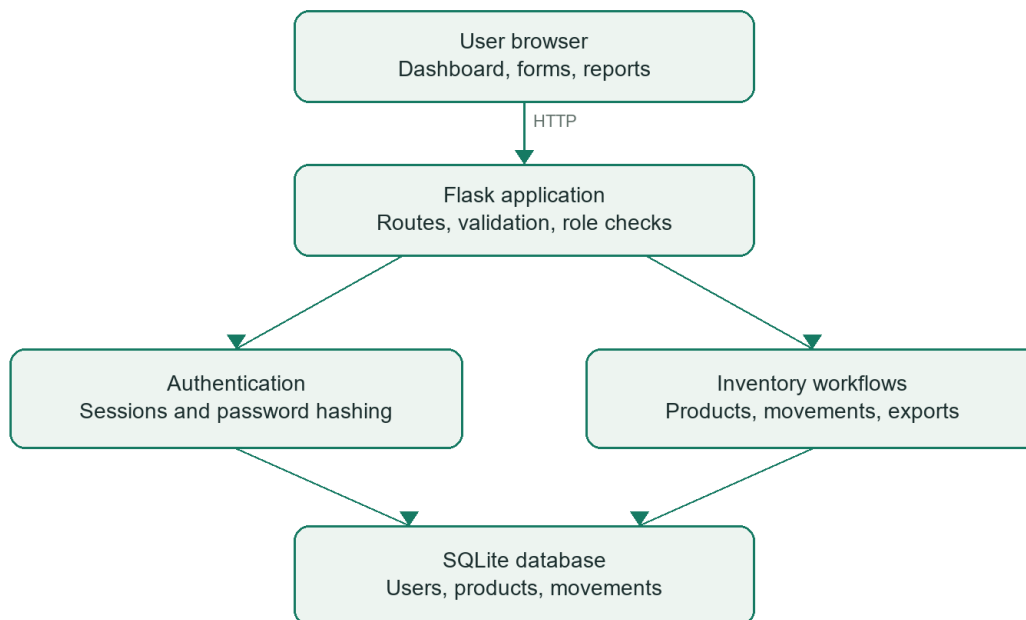


Figure 1: High-level request and response flow of the system.

A user submits a form or requests a page. Flask loads the session user, checks access and form tokens where required, validates input, and queries or updates SQLite. The response is rendered as HTML or delivered as a CSV download.

Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram

Entity	Important attributes
users	id (PK), username (unique), password_hash, role
products	id (PK), sku (unique), name, category, price_cents, quantity, threshold, active
movements	id (PK), product_id (FK), user_id (FK), kind, amount, balance, note, created_at

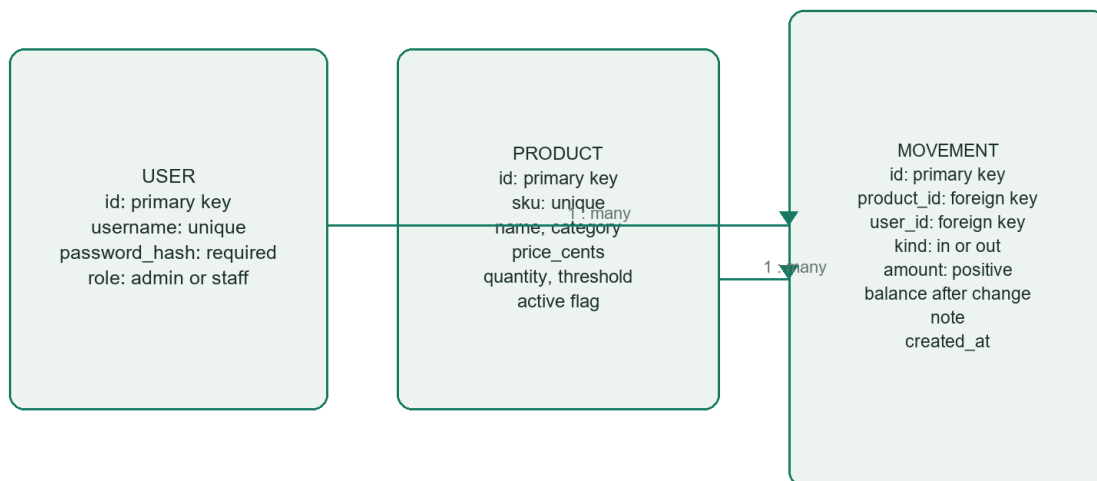


Figure 2. User, product, and stock-movement database model.

Stock Movement Algorithm

1. Authenticate the user and verify the form's CSRF token.

2. Validate product ID, direction, positive integer quantity, and note length.
3. Begin an immediate SQLite transaction to serialize balance updates.
4. Calculate a positive delta for stock-in or a negative delta for stock-out.
5. Update only an active product whose resulting quantity is between zero and the configured maximum.
6. Reject the operation and roll back if no product matches those conditions.
7. Read the resulting balance and insert a movement linked to the current user.
8. Commit both changes together, then display the movement history.

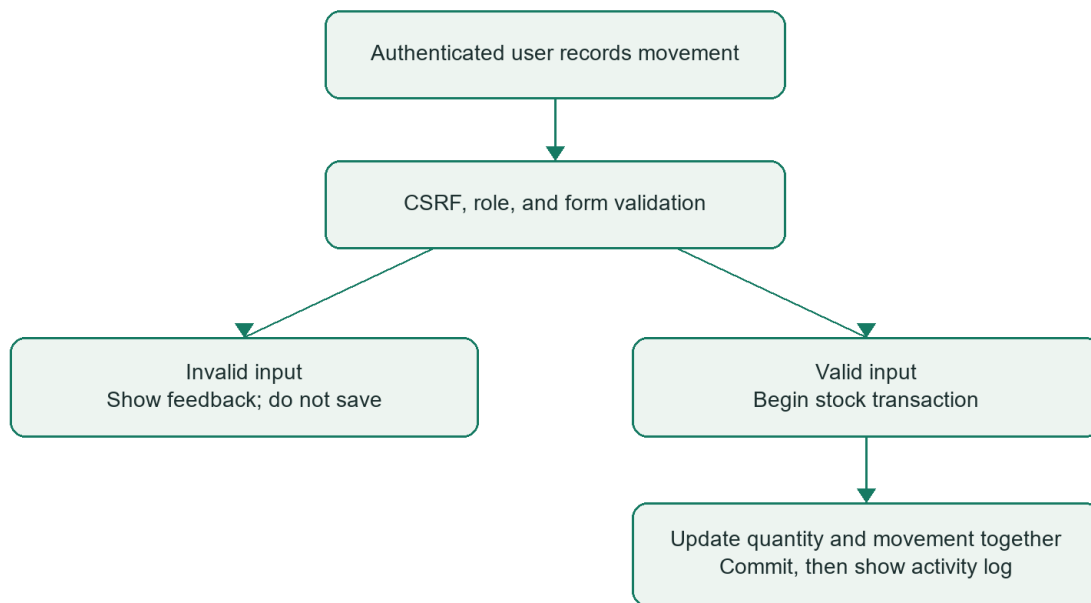


Figure 3. Stock-movement validation and transaction flow.

5.3 Interfaces (if applicable)

Route / interface	Method	Purpose / access
/login	GET / POST	Sign-in form and credential check.
/	GET	Dashboard; authenticated users.
/products	GET	Search/filter active inventory.

Route / interface	Method	Purpose / access
/products/new	GET / POST	Create a product; admin only.
/products/<id>/edit	GET / POST	Edit product details; admin only.
/products/<id>/archive	POST	Archive a zero-stock product; admin only.
/stock	GET / POST	Record stock receipt or issue.
/movements	GET	View permanent activity history.
/users	GET / POST	List/create team accounts; admin only.
/export/products.csv	GET	Download active product inventory.
/logout	POST	Clear the login session.

Setup interfaces: the create-admin CLI command creates an administrator without default credentials; seed-demo adds six sample products only when the product database is empty.

5.4 Application Screenshots

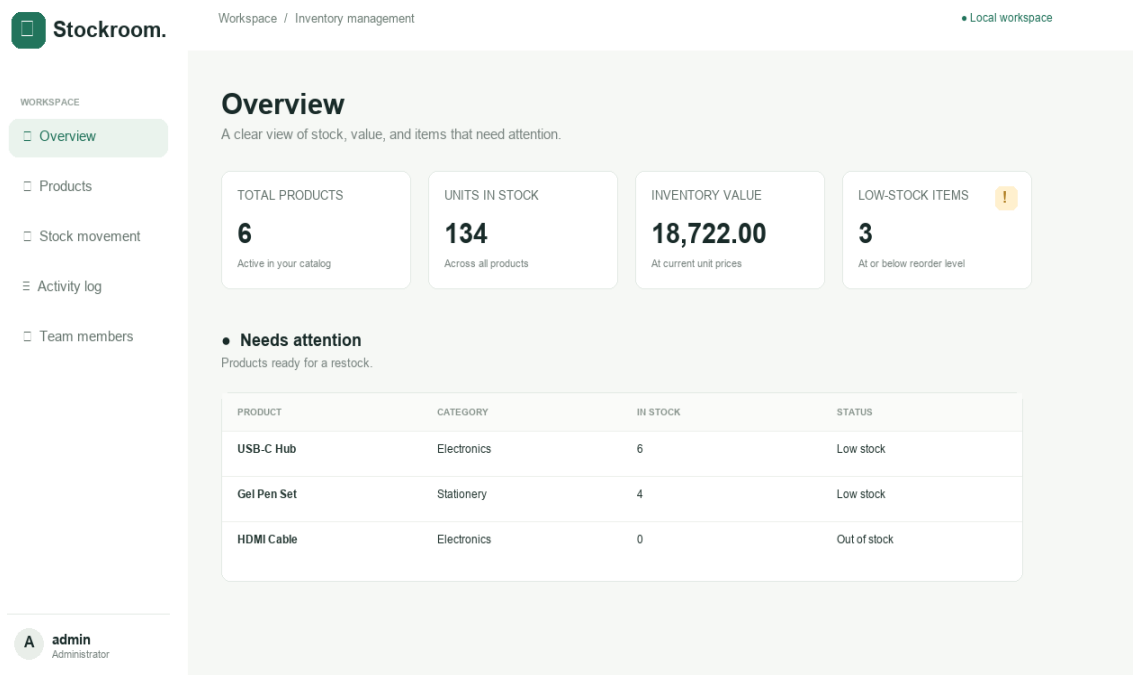


Figure 4. Stockroom dashboard with metrics and low-stock alerts.

Stockroom.

WORKSPACE

Overview

Products

Stock movement

Activity log

Team members

A admin

Administrator

Workspace / Inventory management

Local workspace

Products

Manage your catalog and keep stock levels in view.

+ Add product

Search product, SKU, or category...

PRODUCT / SKU	CATEGORY	UNIT PRICE	STOCK	STATUS
Wireless Mouse / EL-001	Electronics	18.99	34	In stock
USB-C Hub / EL-002	Electronics	34.99	6	Low stock
A5 Notebook / ST-001	Stationery	4.99	72	In stock
Gel Pen Set / ST-002	Stationery	7.99	4	Low stock

Figure 5. Product catalog with searchable stock status.

Stockroom.

WORKSPACE

Overview

Products

Stock movement

Activity log

Team members

A admin

Administrator

Workspace / Inventory management

Local workspace

Stock movement

Every change is recorded with the person and resulting balance.

Record stock movement

Product

USB-C Hub · EL-002 · 6 in stock

Movement type

Stock in — receive inventory

Quantity

10

Reason / reference

Supplier delivery PO-1042

Record movement

Figure 6. Stock receipt form with transaction reason.

Stockroom.

WORKSPACE

Overview

Products

Stock movement

Activity log

Team members

Workspace / Inventory management

Local workspace

Activity log

A permanent record of every stock movement.

PRODUCT	MOVEMENT	BALANCE	RECORDED BY	REASON
USB-C Hub	+10 · Stock in	16	admin	Supplier delivery PQ-1042
Wireless Mouse	+34 · Stock in	34	admin	Opening stock
Gel Pen Set	+4 · Stock in	4	admin	Opening stock
HDMI Cable	+0 · Stock in	0	admin	Opening stock

The latest receipt is now visible with the updated balance.

A admin

Administrator

Figure 7. Permanent stock-movement activity log.

Stockroom.

WORKSPACE

Overview

Products

Stock movement

Activity log

Team members

Workspace / Inventory management

Local workspace

Team members

Admins manage the catalog and accounts. Staff can record stock movements.

USERNAME	ROLE
admin	Admin
warehouse-staff	Staff

Add team member

Username

warehouse-staff

Role

Staff

Create user

A admin

Administrator

Figure 8. Team-member roles and account management

6 Performance Test

Validation focuses on functional correctness and stock integrity. No throughput, latency, memory, or multi-user load benchmark has been completed. The outcomes below should not be interpreted as a production performance claim.

Constraint	Design response	Validation / limitation
Negative inventory	Conditional update plus nonnegative database constraint.	An overdraw is rejected and history/balance stay unchanged.
Partial stock updates	Balance and movement are in one transaction.	Tests verify accepted/rejected movement balances and history.
Unauthorized administration	Server-side admin role checks.	Staff requests to restricted routes return HTTP 403.
Malformed inputs	Server validation and database constraints.	Tests reject invalid prices and movement quantities.
History loss	Movement records are permanent; products are archived.	Tests confirm archived product history remains visible.
Concurrent writes	BEGIN IMMEDIATE and SQLite lock timeout.	Write serialization is implemented; concurrency/load benchmarking remains future work.
Large activity history	Current page loads the full history.	Pagination is recommended before large-scale use.

6.1 Test Plan/ Test Cases

ID	Test method / cases	Observed result
T01	test_login_and_authentication: unauthenticated redirect, bad credentials, login, logout.	Pass
T02	test_staff_permissions: deny admin pages/archival; allow stock, catalog, reporting.	Pass

ID	Test method / cases	Observed result
T03	test_movements_and_rejected_overdraw: receive 10, issue 7, reject issue 4; reject malformed/nonpositive quantities.	Pass; final balance 3, two movements.
T04	test_product_validation_edit_archive_and_history: create/edit, duplicate SKU, invalid prices, zero-stock archival, retained history.	Pass
T05	test_csrf_user_creation_csv_and_templates: token rejection, user validation, CSV formula escaping, page rendering, missing product.	Pass
T06	test_cli: create admin, reject demo seed on nonempty catalog, seed six products on empty catalog.	Pass

6.2 Test Procedure

1. Install dependencies from requirements.txt and use the project directory as the working directory.
2. Run: `python -m unittest discover -s tests -v`
3. Each test creates a temporary SQLite database, prepares users/products, and uses Flask's test client for requests.
4. Assertions check HTTP responses, rendered content, persisted quantities, movement counts, and CLI results.
5. Database connections are closed and temporary test data is cleaned up.
6. Repeat manually in the browser: create a product, receive stock, issue stock, attempt an overdraw, export CSV, and check staff restrictions.

6.3 Performance Outcome

Verified development environment: Python 3.8.10 and Flask 3.0.0 on Windows. The reproducible dependency file specifies Flask 3.0.3 and Werkzeug 3.0.6; the recorded test run used the already-installed environment, not a newly installed dependency environment.

Recorded automated result: Ran 6 tests in 3.207 seconds — OK. This is test-suite duration, not application response latency. Re-run the suite in your submission environment and insert an updated screenshot if its versions or timing differ.

[Insert Figure 7: terminal screenshot of your final passing test run]

The tested scenarios demonstrate rejection of stock overdrafts, preserved balances after invalid requests, protected admin interfaces, and retained history after archival. Responsive layout, browser compatibility, and simultaneous-user behavior require additional manual or dedicated testing; no visual-browser verification is claimed here.

7 My learnings

Personalize this section after reviewing and demonstrating the project. The following learning summary describes the concepts covered by the implementation; keep only statements you can explain from your own experience.

Python and Flask

The project illustrates how route functions process requests, validate forms, render Jinja templates, and redirect after successful writes. The application factory allows isolated test configurations.

Relational data modeling

Separate account, product, and movement tables avoid mixing identity, current state, and historical events. Foreign keys connect movements to their product and actor.

Transactions and business rules

A stock balance must not be changed without a corresponding history event. A transaction makes those changes succeed or fail together, and the conditional update prevents overdrawing stock.

Authentication and security

Password hashes protect stored credentials. Session-based identity, role checks, CSRF validation, parameterized SQL, and template escaping address different parts of application security.

Validation and testing

Unhappy paths such as negative quantities, duplicate identifiers, unauthorized actions, and insufficient stock are as important as successful form submissions.

Documentation and presentation

A reproducible setup guide, small demo scenario, clear limitations, and meaningful screenshots make a project easier to review and maintain.

My personal challenges and solutions: [Describe one or two problems you encountered and how you resolved them.]

8 Future work scope

Enhancement	Expected benefit
Supplier and purchase-order modules	Connect low-stock decisions with ordering and receiving.
Barcode / QR scanning	Reduce manual product selection and entry errors.
Multiple stock locations	Track warehouses, transfers, and per-location availability.
Email alerts	Notify responsible staff outside the dashboard.
History pagination and filters	Keep activity pages usable as the movement table grows.
PostgreSQL and load testing	Evaluate stronger multi-user write workloads with measured evidence.
Account lifecycle controls	Add password reset, account deactivation, and restricted role changes.
Deployment hardening	Production WSGI server, HTTPS, secure cookies, login rate limits, and backups.
Stock valuation reports	Add explicit currency and a defined accounting valuation method.